

Lesson 3: State and Event Handling with useState

Mission Brief

Your Mission Control components can display data, but they're frozen in time — nothing on screen changes when a crew member clicks a button. Real spacecraft dashboards need live counters, toggles, and forms that react the instant someone interacts with them. Today you give your components **memory** using React's `useState` hook, and teach them to respond to clicks, typing, and other events.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain why props alone can't make a component change over time
- Use the `useState` hook to add local, changeable state to a component
- Update state correctly using a setter function (never mutating directly)
- Attach event handlers like `onClick`, `onChange`, and `onSubmit` to JSX elements
- Explain why calling a state setter triggers a re-render

What You'll Build

A `LaunchCounter` component with a live count and Increase/Decrease/Reset buttons, plus a `CrewToggle` component that flips a crew member's status between "On Duty" and "Off Duty" when clicked.

Prerequisites

- Comfortable creating and composing components (Lesson 1)
- Comfortable with JSX, props, and `.map()` rendering (Lesson 2)

Key Concepts

- The `useState` hook and its array-destructuring syntax
- State vs. props (who owns the data, and can it change)
- Immutability: replacing state instead of mutating it
- Event handlers in JSX (`onClick`, `onChange`, `onSubmit`)
- Re-renders: what actually happens when state changes

Concept Explanation

Props are data a component *receives* from its parent — the component itself cannot change them. But plenty of things in a real UI need to change based on user interaction: a counter, a toggle switch, the text in a search box. For that, a component needs its own **state**: data it owns and can update over its lifetime.

React gives you this with the `useState` hook:

```
import { useState } from "react";

function LaunchCounter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>T-minus adjustments: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
      <button onClick={() => setCount(count - 1)}>Decrease</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

`useState(0)` does two things at once: it creates a piece of state initialized to `0`, and it returns an array with exactly two items — the current value (`count`) and a function to change it (`setCount`). The array-destructuring syntax `const [count, setCount] = useState(0)` is just a convenient way to name both items at once; you could call them anything, but `thing` / `setThing` is the near-universal convention.

The crucial rule: **you never modify state directly**. Writing `count = count + 1` does nothing useful and will not update the screen — React has no way of knowing you changed it. You must always call the setter function, `setCount(...)`, because calling the setter is what tells React "something changed, please re-render this component." When `setCount` runs, React schedules the component function to run again from the top, this time with `count` holding the new value, and this fresh output is what replaces the old UI on screen (efficiently, only where it actually differs).

Event handlers connect user actions to state changes. In plain HTML you might write `onclick="doThing()"` as a string; in JSX, event props are camelCased and take an actual JavaScript function, not a string: `onClick={() => setCount(count + 1)}`. The arrow function matters — `onClick={setCount(count + 1)}` (without the arrow) would call `setCount` immediately during render instead of waiting for a click, causing an infinite loop of re-renders. Wrapping it in `() => ...` creates a function that only runs *when the click happens*.

Toggling booleans follows the same pattern:

```
function CrewToggle({ initialOnDuty = true }) {
  const [onDuty, setOnDuty] = useState(initialOnDuty);

  return (
    <button onClick={() => setOnDuty(!onDuty)}>
      {onDuty ? "● On Duty" : "● Off Duty"} – click to change
    </button>
  );
}
```

For text inputs, the `onChange` event fires on every keystroke, and you typically read the new value from `event.target.value`:

```
function CallsignInput() {
  const [callsign, setCallsign] = useState("");

  return (
    <div>
      <input
        value={callsign}
        onChange={(e) => setCallsign(e.target.value)}
        placeholder="Enter your callsign"
      />
      <p>Hello, {callsign || "stranger"}!</p>
    </div>
  );
}
```

Notice this input's `value` is set from state, and every change updates that same state — this pattern is called a **controlled component**, and it's the standard way to handle form fields in React (you'll go deeper on forms in Lesson 5).

One more subtlety: state updates based on the *previous* state value should use the updater-function form of the setter, `setCount(prev => prev + 1)`, rather than `setCount(count + 1)`, especially when multiple updates might happen close together. The updater form guarantees React hands you the truly latest value rather than one that might be stale from an earlier render.

Guided Walkthrough

1. In your Vite React project, create `src/components/LaunchCounter.jsx` with the counter code above (Increase / Decrease / Reset).
2. Import and render `<LaunchCounter />` inside `App.jsx` and confirm clicking each button updates the number on screen instantly.

3. Create `src/components/CrewToggle.jsx` with the on-duty/off-duty toggle. Render three or four of them with different `initialOnDuty` props and confirm each toggles independently — this proves each component instance has its *own* separate state.
4. Add a text input using the controlled-component pattern (`CallsignInput`) and confirm the greeting updates on every keystroke.
5. Deliberately break the counter by changing `setCount(count + 1)` to `count = count + 1` (no setter call) and observe that clicking now does nothing — then fix it back, cementing why the setter call is required.

Common Bugs & Debugging Tips

- Forgetting the arrow function (`onClick={setCount(count + 1)}` instead of `onClick={() => setCount(count + 1)}`) either fires immediately on render or creates an infinite loop — React DevTools' console will often show "Too many re-renders."
- Directly mutating state (`someArray.push(item)`) then calling `setSomeArray(someArray)`) does not trigger a re-render, because the array reference didn't change — always create a new array/object, e.g. `setSomeArray([...someArray, item])` .
- Typing into an `<input>` that shows no changes is almost always a missing `onChange` handler, or a handler that doesn't call the setter.
- If state seems to be "one step behind" after rapid clicks, switch to the updater-function form: `setCount(prev => prev + 1)` .

Check Your Understanding

1. What are the two values returned by `useState` , in order?
2. Why does `count = count + 1` fail to update the screen, while `setCount(count + 1)` works?
3. What does it mean for an `<input>` to be a "controlled component"?
4. When should you prefer `setCount(prev => prev + 1)` over `setCount(count + 1)` ?
5. If you render the same component twice on a page, do they share state or have separate state? Why?

Mini Quiz + Answer Key

- Q1.** What does `const [count, setCount] = useState(0)` set the initial value of `count` to? A) `undefined` B) `0` C) `null` D) `"0"`
- Q2.** Which correctly attaches a click handler that increases state by one? A) `onClick="setCount(count+1)"` B) `onClick={setCount(count+1)}` C) `onClick={() => setCount(count + 1)}` D) `onclick={() => setCount(count + 1)}`
- Q3.** Why must you use `setCount` instead of reassigning the `count` variable? A) `count` is a constant declared with `const` and reassigning it doesn't trigger a re-render B) JavaScript forbids reassignment entirely C) `count` is a string D) There is no difference

Q4. What is a "controlled component"? A) A component with no props B) An input whose value is driven by React state and updated via `onChange` C) A component that controls its parent D) A class component

Answer Key: 1-B, 2-A, 3-A, 4-B

Lesson Recap

You learned that props are read-only data from a parent, while state is data a component owns and can change over time via `useState`. You attached event handlers (`onClick`, `onChange`) that call state setters, learned why direct mutation doesn't work, and built controlled inputs — the foundation for every interactive feature in Projects 07-09.

Homework

Extend `LaunchCounter` into a `FuelGauge` component: state starts at `100`, a "Burn Fuel" button decreases it by 10 (never below 0), a "Refuel" button resets it to 100, and the displayed number turns red (inline style or conditional `className`) whenever fuel drops below 20.

Portfolio Project Connection

Project 07's "Add to Cart" quantity stepper, Project 08's like-button toggle and comment-input field, and Project 09's contact form all rely directly on `useState` and event handlers exactly as covered in this lesson.